

Numeric Data-Types

A table supports all the standard SQL and MySQL numerical data. The data-types in this category are predominantly used to reflect the quantitative value of the information being stored. A table can use the numeric data-types to specify and quantify the value of stored data. For example, when the numerical value of data is an integer, it is defined under the INTEGER data-type.

MySQL DATA TYPES			
DATE TYPE	SPEC	DATE TYPE	SPEC
CHAR	String (0 - 255)	INT	Integer (-2147483648 to 2147483647)
VARCHAR	String (0 - 255)	BIGINT	Integer (-9223372036854775808 to 9223372036854775807)
TINYTEXT	String (0 - 255)	FLOAT	Decimal (precise to 23 digits)
TEXT	String (0 - 65535)	DOUBLE	Decimal (24 to 53 digits)
BLOB	String (0 - 65535)	DECIMAL	"DOUBLE" stored as string
MEDIUMTEXT	String (0 - 16777215)	DATE	YYYY-MM-DD
MEDIUMBLOB	String (0 - 16777215)	DATETIME	YYYY-MM-DD HH:MM:SS
LONGTEXT	String (0 - 4294967295)	TIMESTAMP	YYYYMMDDHHMMSS
LOBLOB	String (0 - 4294967295)	TIME	HH:MM:SS
TINYINT	Integer (-128 to 127)	ENUM	One of preset options
SMALLINT	Integer (-32768 to 32767)	SET	Selection of preset options
MEDIUMINT	Integer (-8388608 to 8388607)	BOOLEAN	TINYINT(1)

Source: <http://www.mysqltutorial.org>

All the data-types MySQL supports

A MySQL table also supports data types such as DOUBLE PRECISION, FLOAT, and REAL. In most tables, the INTEGER data-type is represented with its acronym “INT”. The non-standard extension of DOUBLE PRECISION data-type is represented with a synonym called “DOUBLE”. In this category, the integer data-types are further classified into INT, BIGINT, SMALLINT, TINYINT, and MEDIUMINT, based on the data’s byte length and the maximum and minimum value assigned to it. The floating-point data-types, on the other hand, are bifurcated into two classes - DOUBLE and FLOAT. The current MySQL tables also support the non-standard variations of floating-point data such as REAL(M,D), FLOAT (M,D) and DOUBLE PRECISION(M,D).

You can also distinguish the fixed-point numeric data-types into DECIMAL and NUMERIC. Similarly, you can use BIT(N) to store the bit-field values of these numeric data-types, where the value of “N” can range from 1 to 128.